

Exercice 10 — Cascade et conflits

🕒 **DURÉE** : ● 15 min | ● 25 min | ● 30 min 🌸 **Niveau Bloom** : Appliquer (3) / Analyser (4) / Évaluer (5) 📖 **Prérequis** : Leçons IX (Sélecteurs avancés) et X (Cascade et héritage)

Objectif

Calculer la spécificité d'un sélecteur, **prédire** laquelle gagne en cas de conflit, **diagnostiquer** un bug de cascade, et **résoudre** en choisissant la règle avec la spécificité minimale viable — sans **!important**.

● Échauffement

Partie 1 — Mémoriser le tableau de spécificité

Recopier dans un tableau et mémoriser :

Sélecteur	Spécificité (ID, classe, type)	Total
p	0, 0, 1	1
.card	0, 1, 0	10
ul li.active	0, 1, 2	12
#footer	1, 0, 0	100

👉 Un ID bat **TOUJOURS** toutes les classes du monde. #header (100) > .nav .menu a:hover.active (41).

Méthode de calcul :

- Compter les IDs (`#...`) → 1ère colonne (× 100).
- Compter les classes (`.xxx`), attributs (`[xxx]`), pseudo-classes (`:xxx`) → 2ème colonne (× 10).
- Compter les types (`div` , `p`) et pseudo-éléments (`::before`) → 3ème colonne (× 1).
- Combinateurs (`>` , `+` , `~` , `,`) : **aucun impact** sur la spécificité.
- `*` (sélecteur universel) : spécificité 0.

Partie 2 — Observer un conflit en direct

Recopier ce HTML dans `cascade.html` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main id="content">
    <p class="intro">Paragraphe intro avec une <a href="#" class="link-cta">action</a>.</p>
    <p>Paragraphe normal avec un <a href="#">lien standard</a>.</p>
  </main>
</body>
</html>
```

Et ce CSS dans `style.css` :

```
/* Règle 1 – type, spécificité 0,0,1 */
a {
  color: blue;
}

/* Règle 2 – classe, spécificité 0,1,0 */
.link-cta {
  color: orange;
}

/* Règle 3 – id descendant + type, spécificité 1,0,1 */
#content a {
  color: green;
}
```

Ouvrir dans le navigateur. **Observer** :

- Le lien `.link-cta` apparaît en **vert** (et non orange comme on pourrait croire).
- Le lien standard apparaît également en vert.
- Pourquoi ? Calculer mentalement la spécificité de chaque sélecteur qui cible le `.link-cta` :
 - `a` → 0,0,1
 - `.link-cta` → 0,1,0
 - `#content a` → 1,0,1 → **gagne** (un id pèse 100, plus que 10 classes).

Vérifier dans DevTools : panneau Styles, les règles non appliquées sont **barrées**. Le panneau **Computed** → **color** doit afficher **green**.

● Normal — À vous de jouer

Partie A — Calculer la spécificité

Remplir le tableau. Vérifier dans la correction.

Sélecteur	Spécificité
<code>p</code>	-----
<code>.card</code>	-----
<code>#footer</code>	-----
<code>ul li.active</code>	-----
<code>nav > ul > li a:hover</code>	-----
<code>#app .sidebar .menu a</code>	-----
<code>input[type="email"]:focus</code>	-----
<code>body.logged-in .header</code>	-----

Partie B — Débugger un fichier

Voici un fichier `bug.html` à débbugger :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <style>
    body { color: black; }

    .article { color: gray; }

    article p { color: darkblue; }

    .article .titre { color: red !important; }

    #hero .titre { color: purple; }

    [data-theme="dark"] .titre { color: white; }
  </style>
</head>
<body data-theme="dark">
  <article class="article" id="hero">
    <h2 class="titre">Le titre de l'article</h2>
    <p>Un paragraphe de l'article.</p>
  </article>
</body>
</html>
```

Questions :

1. De quelle couleur est le **titre** rendu à l'écran ? Justifier en calculant la spécificité de chaque règle qui le cible.
2. Quelle est la couleur du **paragraphe** ?
3. **Refactor** : réécrire ce CSS pour que :
 - Le titre soit vert (`#0a8a4a`).
 - Le paragraphe reste dans sa couleur actuelle.
 - **Aucun** `!important` .
 - Spécificité totale minimale (idéalement, classes uniquement).

Livrable : un fichier `bug-fixed.css` réécrit + un commentaire en tête expliquant les choix.

● Défi — Explorer `@layer` (CSS Cascade Layers)

Les **cascade layers** (`@layer`) permettent de **regrouper** des règles CSS en couches et de leur donner une priorité **globale** indépendamment de la spécificité.

Lire la spec MDN : **CSS Cascade Layers** .

Exercice : refactor un CSS existant en utilisant 4 layers :

```
@layer reset, base, composants, utilities;

@layer reset {
  /* reset universel, normalize */
}

@layer base {
  /* typo, couleurs, layout de base */
}

@layer composants {
  /* .article, .btn-cta, .nav, .card */
}

@layer utilities {
  /* .text-center, .mb-2, .visually-hidden */
}
```

Avantages à observer :

- Une règle dans `@layer base` ne peut **jamais** écraser une règle dans `@layer composants`, **même avec une spécificité plus haute**.
- L'ordre `reset, base, composants, utilities` indique : **plus à droite = plus prioritaire**.

Questions :

1. Une règle `#hero .titre { color: red; }` dans `@layer base` est-elle plus prioritaire que `.titre { color: blue; }` dans `@layer composants` ? Pourquoi ?
2. Quelle compatibilité navigateur ? (caniuse.com/css-cascade-layers)
3. Quand utiliser `@layer` plutôt que la spécificité classique ?

Livrable : `digiblog/style-layers.css` refactorisé + section dans `AUDIT-J2.md` qui répond aux 3 questions.